Name :N. Sai Sreenu
Reg No: 192365001

C05-AT2
MLA0201-Fundamentals Of Machine Learning

Reinforcement Learning & Probabilistic Algorithms

Pseudocode Solutions

Question 1: Monte Carlo Sampling for Probability Estimation

Problem: A logistics company wants to estimate the probability of on-time deliveries under uncertain traffic conditions using Monte Carlo Sampling.

Task Requirements

- Generate random samples representing delivery conditions.
- Simulate multiple delivery trials.
- Estimate the probability of on-time delivery.
- Display the estimated probability after all simulations.

Algorithm

Pseudocode

```
BEGIN MonteCarlo_OnTimeDelivery
  INPUT: N  // total number of simulation trials
  SET on_time_count = 0

  FOR trial = 1 TO N DO
      // Step 1: Generate a random sample representing delivery conditions
      traffic_condition = RANDOM_VALUE(0, 1)      // 0 = clear, 1 = heavy traffic
      weather_condition  = RANDOM_VALUE(0, 1)
      delay = f(traffic_condition, weather_condition)  // simulated delay function

      // Step 2: Simulate the delivery trial
      actual_delivery_time = base_delivery_time + delay

      // Step 3: Check if delivery is on time
      IF actual_delivery_time <= scheduled_delivery_time THEN
```

Pseudocode

```
            on_time_count = on_time_count + 1
        END IF
    END FOR

    // Step 4: Estimate probability of on-time delivery
    P_on_time = on_time_count / N

    // Step 5: Display result
    PRINT "Estimated Probability of On-Time Delivery = ", P_on_time
END MonteCarlo_OnTimeDelivery
```

Question 2: ε-Greedy Algorithm for K-Armed Bandit

Problem: An online streaming platform recommends one of several movies to maximize user engagement. The platform must balance exploration and exploitation.

Task Requirements

- Initialize rewards for all movie recommendations.
- Implement the ε-greedy exploration strategy.
- Update reward estimates after each user interaction.
- Identify the movie with the highest expected reward.

Algorithm

Pseudocode

```
BEGIN Epsilon_Greedy_Bandit
    INPUT: K            // number of movies (arms)
    INPUT: epsilon      // exploration probability (0 <= epsilon <= 1)
    INPUT: T            // total number of user interactions

    // Step 1: Initialize reward estimates and selection counts
    FOR each movie i = 1 TO K DO
        Q[i] = 0        // estimated reward for movie i
        count[i] = 0    // number of times movie i was recommended
    END FOR
```

Pseudocode

```
    FOR t = 1 TO T DO
        // Step 2: epsilon-greedy action selection
        random_number = RANDOM_VALUE(0, 1)
        IF random_number < epsilon THEN
            action = RANDOM_MOVIE(1, K)          // Explore: pick a random movie
        ELSE
            action = ARGMAX(Q[1..K])             // Exploit: pick best-known movie
        END IF

        // Step 3: Observe reward from user interaction (e.g., watch time, rating)
        reward = OBSERVE_USER_REWARD(action)

        // Step 4: Update reward estimate (incremental average)
        count[action] = count[action] + 1
        Q[action] = Q[action] + (reward - Q[action]) / count[action]
    END FOR

    // Step 5: Identify movie with highest expected reward
    best_movie = ARGMAX(Q[1..K])
    PRINT "Movie with highest expected reward = ", best_movie
END Epsilon_Greedy_Bandit
```

Question 3: Value Iteration for Robot Path Planning

Problem: A warehouse robot must determine the optimal path to reach a destination while maximizing rewards and avoiding obstacles.

Task Requirements

- Initialize the value function for all states.
- Update state values using the Bellman optimality equation.
- Repeat the process until convergence.
- Output the optimal value function and policy.

Algorithm

Pseudocode

```
BEGIN Value_Iteration_RobotPathPlanning
  INPUT: S            // set of all states (grid cells)
  INPUT: A            // set of possible actions (up, down, left, right)
  INPUT: P(s'|s,a)    // transition probabilities
  INPUT: R(s,a,s')    // reward function
  INPUT: gamma        // discount factor (0 <= gamma < 1)
  INPUT: theta        // small threshold for convergence

  // Step 1: Initialize value function for all states
  FOR each state s in S DO
      V[s] = 0
  END FOR

  REPEAT
      delta = 0
      FOR each state s in S DO
          v_old = V[s]

          // Step 2: Bellman optimality update
          V[s] = MAX over a in A of
                  SUM over s' of P(s'|s,a) * (R(s,a,s') + gamma * V[s'])

          delta = MAX(delta, |v_old - V[s]|)
      END FOR
  UNTIL delta < theta       // Step 3: repeat until convergence

  // Step 4: Derive the optimal policy from the converged values
  FOR each state s in S DO
      policy[s] = ARGMAX over a in A of
                  SUM over s' of P(s'|s,a) * (R(s,a,s') + gamma * V[s'])
  END FOR

  PRINT "Optimal Value Function = ", V
  PRINT "Optimal Policy = ", policy
```

Pseudocode

```
END Value_Iteration_RobotPathPlanning
```

Question 4: Temporal Difference (TD) Learning for Game Score Prediction

Problem: A game AI agent learns the value of each game state based on rewards received after every move.

Task Requirements

- Initialize state-value estimates.
- Observe the current state, reward, and next state.
- Update the state value using the Temporal Difference (TD) learning rule.
- Continue until all game episodes are completed.

Algorithm

Pseudocode

```
BEGIN TD_Learning_GameScorePrediction
  INPUT: alpha         // learning rate (0 < alpha <= 1)
  INPUT: gamma         // discount factor (0 <= gamma < 1)
  INPUT: num_episodes   // total number of game episodes

  // Step 1: Initialize state-value estimates
  FOR each state s DO
      V[s] = 0   // or small random values
  END FOR

  FOR episode = 1 TO num_episodes DO
      state = INITIAL_STATE()

      WHILE state is NOT terminal DO
          // Step 2: Observe current state, take action, get reward and next state
          action = SELECT_ACTION(state)
          reward, next_state = TAKE_ACTION(state, action)

          // Step 3: TD learning update rule
```

Pseudocode

```
            V[state] = V[state] + alpha * (reward + gamma * V[next_state] - V[state])

            state = next_state
        END WHILE
    END FOR   // Step 4: continue until all episodes are completed

    PRINT "Learned State-Value Estimates = ", V
END TD_Learning_GameScorePrediction
```

Question 5: Policy Iteration for Autonomous Drone Navigation

Problem: An autonomous drone must learn the optimal navigation policy to deliver packages while minimizing energy consumption.

Task Requirements

- Initialize a random policy.
- Perform policy evaluation to compute state values.
- Improve the policy by selecting the best action for each state.
- Repeat until the policy becomes stable and output the optimal policy.

Algorithm

Pseudocode

```
BEGIN Policy_Iteration_DroneNavigation
    INPUT: S, A, P(s'|s,a), R(s,a,s'), gamma, theta

    // Step 1: Initialize a random policy and value function
    FOR each state s in S DO
        policy[s] = RANDOM_ACTION(A)
        V[s] = 0
    END FOR

    policy_stable = FALSE

    WHILE NOT policy_stable DO
```

Pseudocode

```
        // Step 2: Policy Evaluation
        REPEAT
            delta = 0
            FOR each state s in S DO
                v_old = V[s]
                a = policy[s]
                V[s] = SUM over s' of P(s'|s,a) * (R(s,a,s') + gamma * V[s'])
                delta = MAX(delta, |v_old - V[s]|)
            END FOR
        UNTIL delta < theta

        // Step 3: Policy Improvement
        policy_stable = TRUE
        FOR each state s in S DO
            old_action = policy[s]
            policy[s] = ARGMAX over a in A of
                        SUM over s' of P(s'|s,a) * (R(s,a,s') + gamma * V[s'])
            IF old_action != policy[s] THEN
                policy_stable = FALSE
            END IF
        END FOR
    END WHILE   // Step 4: repeat until policy is stable

    PRINT "Optimal Policy = ", policy
    PRINT "Optimal Value Function = ", V
END Policy_Iteration_DroneNavigation
```